RoundFlow Backend — Developer Handoff Guide

Project: RoundFlow (Agentum AI) — field-service SaaS for UK window-cleaning businesses **Repo:** `github.com/agenticgeek/roundflow-backend` · default/integration branch: `dev` **Prepared:** 2026-10-13 · for the incoming backend developer

> Read this top to bottom once, then do the **Local setup** (§5) yourself before touching anything. This guide follows the standard KT checklist and calls out the RoundFlow-specific landmines (QA not set up, prod not set up, Gotenberg on prod, GHL marketplace app in draft).

---

0. The 10-minute orientation

- **Backend = the system of record.** Node/Express + TypeScript, Prisma 6, Supabase Postgres. A separate React frontend (admin web) and a React Native **technician app** consume this API. GHL, GoCardless and Stripe are *external utilities*, not the platform.
- **Multi-tenant, schema-per-tenant.** One Postgres instance; each business (tenant) gets its own schema `t_<hex>`. The `public` schema holds only `Profile`, `Tenant`, `TenantInvite`.
- **Auth is Supabase** (ES256 JWTs verified via JWKS). The backend has **no** login/signup/password endpoints — those are Supabase, client-side.
- **Deploys:** GitHub Actions → ECR → **EC2 (dev + qa, `docker run`)** and **ECS (prod)**. Push to `dev` / `qa` / `prod` branches deploys to the matching env.
- **Status in one line:** everything built in Oct 2026 (customer messaging, invoice PDF, reports export, per-user preferences, the mobile-app gap fixes) is **live on DEV only**. **QA is dormant. Prod is not set up.**

---

1. Access & ownership (do this first)

Area	What / where	Notes
Code	`github.com/agenticgeek/roundflow-backend`	Get write access. Branch model below. Set branch protection on `prod` (and ideally `dev`).
AWS	Account `574667603029`, region `eu-north-1`	ECR repo `roundflow`; EC2 (dev+qa shared box); ECS (prod, to set up). Get an IAM user — **don't** use the root account.
EC2 (dev/qa box)	Host `51.20.108.133`, user `ec2-user`, key name `roundflow-ec2`	SSH key is `roundflow-ec2.pem` — get it via password manager. This one box runs **both** `roundflow-dev` (:3000) and `roundflow-qa` (:3001).
Supabase	Dev project exists. **QA + Prod projects need creating** (see §5).	QA project is to be created **on Mark Coates' email**. This is the system-of-record DB **and** the auth provider.
GoHighLevel	Agency account + **marketplace app (currently in DRAFT)**	The app's OAuth must be finished before tenants can connect (see §4). Workflows to build: `docs/GHL_AUTOMATION_OWNER_HANDOFF.md`.
GoCardless / Stripe	GoCardless dashboard; Stripe **Connect** platform	Sandbox + live keys. Payment connect is blocked on these accounts existing (see §4/§7).
Google Cloud Console	**On Mark Coates' email**	Hosts the **Google OAuth** client (used by Supabase Auth for "Sign in with Google") and the **Google Maps API key** the frontend needs (Round Planner map / dashboard GPS). The backend itself does not call Google.
Expo	Push notifications (optional `EXPO_ACCESS_TOKEN`)	The mobile app is assumed to be Expo; push delivery goes through Expo's push API.
Domains / DNS	`api.roundflow.ai` (prod API), `api-dev.roundflow.ai` (dev API), `dev.roundflow.ai` + `www.roundflow.ai` (frontend, AWS Amplify)	Confirm where DNS is managed.

Secrets inventory (every env var the backend reads)

Stored in: **host files** `/home/ec2-user/.env.dev` and `.env.qa` on the EC2 box (dev/qa), the **ECS task definition** (prod), and **GitHub Actions secrets** (CI: `DATABASE_URL`, `DIRECT_URL`, `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `EC2_HOST`, `EC2_SSH_KEY`, and prod-only `ECS_TASK_DEFINITION`, `ECS_SERVICE`, `ECS_CLUSTER`). `.env.example` in the repo is the committed template.

Var	Purpose
`DATABASE_URL`	Supabase pooler, port 6543 (`?pgbouncer=true`) — app runtime
`DIRECT_URL`	Supabase direct, port 5432 — Prisma migrations + tenant provisioning
`SUPABASE_ISSUER`	`https://<ref>.supabase.co/auth/v1` — JWT issuer + JWKS base (ES256 verify)
`SUPABASE_URL`	Storage signed-upload URLs + public file URLs
`SUPABASE_SERVICE_ROLE_KEY`	Supabase Auth Admin API (staff invite emails). **Server-only, never frontend.**
`VISIT_PHOTOS_BUCKET`	Supabase Storage bucket (`visit-photos`) — visit photos, avatars, chat images
`INVITE_BASE_URL`	Frontend base for `/accept-invite?token=` links
`FRONTEND_URL`	**Comma-separated** allowed CORS origins; the **first** is canonical for OAuth/invite redirects
`PUBLIC_API_URL`	Public API base — builds OAuth callback URLs (Stripe/GHL)
`ENCRYPTION_KEY`	**64-char hex (32 bytes).** AES key that encrypts stored GHL/GoCardless/Stripe tokens at rest. **If this is lost or changed, every tenant's provider connections break and must be re-authorized.** Guard it.
`GHL_CLIENT_ID` , `GHL_CLIENT_SECRET`	GHL marketplace app OAuth (app-level)
`STRIPE_CLIENT_ID` , `STRIPE_SECRET_KEY`	Stripe Connect platform credentials
`GOTENBERG_URL`	HTML→PDF render service for invoices. Unset ⇒ `/invoices/:id/pdf` returns **501**.
`EXPO_ACCESS_TOKEN`	*(optional)* Expo push auth
`PORT`	App port (default 3000)

> Leftover/ignore: `RAILWAY_PUBLIC_DOMAIN` is a **leftover from a previous Railway deployment** (the app is now on AWS). `TENANT_MIGRATION_URL` / various `PRISMA_*` / `DOTENV_*` are tooling.

---

2. Architecture & documentation

- **Stack:** Node 22, Express, TypeScript, **Prisma 6.19.3** (deliberately *not* 7 — see ARCHITECTURE.md), Supabase Postgres, `vitest` (tests), `tsx` (run/dev), `node-cron` (jobs). HTTP-JSON API only (no HTML rendering).
- **Two Prisma clients** — this trips people up:
- **Public** schema → `prisma/schema.prisma` → `@prisma/client` . Migrations in `prisma/migrations/` , applied by CI `prisma migrate deploy` .
- **Tenant** schema → `prisma/tenant/schema.prisma` → `src/generated/tenant-client` . Migrations in `prisma/tenant/migrations/` as **hand-written** `migration.sql` , applied **at app startup** per tenant schema. Regenerate with `npx prisma generate --schema prisma/tenant/schema.prisma` .
- **Folder structure (`src/`):** `routes/` (thin HTTP handlers) · `services/` (business logic + DB) · `middleware/` (`requireAuth` , `requireTenantAccess` , `requireRole`) · `integrations/` (`ghl/` , `gocardless/` , `stripe/` , `pdf/`)

· `jobs/` (`visitGeneration`, `outboxWorker`) · `mobile/` (the technician-app API: `routes.ts` + `services/`) · `lib/` (`prisma`, `tenant-prisma-manager`, `tenant-provisioning`, `crypto`, `storage`, `http`, `app-error`).

- **Docs to read (and their caveats):**
- `docs/ARCHITECTURE.md` — accurate on multi-tenancy + **correctly describes one GHL sub-account per tenant** (the KT's "single shared GHL account" concern is already reconciled here). ⚠️ its **overview still says "deployed on Railway" — stale, it's AWS now.**
- `PROGRESS.md` — living log; **stale before the 2026-10-13 entries** (the whole Jul→Oct build-out isn't logged). Code is the source of truth.
- `docs/Handoffs/` — per-feature handoffs + the recent gap responses: `FRONTEND_GAPS_RESPONSE_2026-10-13.md`, `MOBILE_GAPS_AUDIT_2026-10-13.md`, `MOBILE_GAPS_RESPONSE_2026-10-13.md`.
- `docs/GHL_AUTOMATION_OWNER_HANDOFF.md` — the GHL custom fields, tags, and **5 workflows that must be built** in GHL.
- `docs/designFindings.md` — frontend/screen source of truth.

---

3. Database

- **Supabase-hosted Postgres**, two connection URLs: pooler `:6543` (`?pgbouncer=true`) for runtime, direct `:5432` for migrations/provisioning (PgBouncer transaction mode can't run migration DDL).
- **Multi-tenancy (schema-per-tenant).** Resolution on every request: `JWT → supabaseUserId → Profile.tenantId → Tenant.schemaName → LRU-cached PrismaClient`. `requireAuth` sets `req.user`; `requireTenantAccess` sets `req.profile` (public Profile) + `req.tenantPrisma` (tenant-scoped client). All tenant queries go through `req.tenantPrisma`, so isolation is by connection/schema, not a `tenantId` column.
- **Migrations:**
- **Public:** add a folder under `prisma/migrations/<ts>_name/migration.sql`, update `prisma/schema.prisma`, `npx prisma generate`. CI runs `prisma migrate deploy`.
- **Tenant:** add `prisma/tenant/migrations/<ts>_name/migration.sql` (hand-written, idempotent — use `ADD COLUMN IF NOT EXISTS` / `CREATE TABLE IF NOT EXISTS`; put `ALTER TYPE ... ADD VALUE` in its **own** migration). Update `prisma/tenant/schema.prisma` + regenerate. Applied automatically at startup by `migrateAllTenantSchemas()` (`src/lib/tenant-provisioning.ts`), tracked per schema in a `_migrations` table, each in its own transaction with `search_path` set. **The Docker build copies `prisma/tenant` → `dist/prisma-tenant`; the runtime reads migrations from there.**
- **Rollback:** there are no down-migrations. To revert, write a new forward migration that undoes the change.
- **Seed:** `prisma/seed.ts` (`npm run seed`) — dev only, idempotent (fixed IDs + upserts). Creates a demo tenant's basics.
- **Backups:** Supabase provides automatic backups per project. **Action item: confirm the backup schedule for the prod project once it exists, and do one test restore together.**
- **Manual prod data fixes:** none yet (prod isn't live).

---

4. APIs & integrations

- **Auth:** Supabase ES256 JWT, verified via JWKS (`src/middleware/requireAuth.ts`, hard-requires `SUPABASE_ISSUER` at import). App roles `ADMIN` / `MANAGER` / `TECHNICIAN` live on `Profile.role` (not the JWT).

`requireRole` / `requireBusinessAccess()` gate mutations to ADMIN/MANAGER. Password reset / OAuth / magic-link are all Supabase-side. The technician app uses the **same** JWT.

- **GHL (GoHighLevel):**
- **One sub-account (location) per tenant.** OAuth via the **marketplace app** (`GHL_CLIENT_ID` / `SECRET` , `src/integrations/ghl/oauth.ts`). Per-tenant access/refresh tokens are **encrypted at rest** (`ENCRYPTION_KEY`) on `BusinessSettings` .
- Outbound messaging: `sendGhlMessage` (GHL conversations). Contact sync + tags (`applyGhlTags`). An **outbox worker** (`src/jobs/outboxWorker.ts` , every 30s) drains `IntegrationOutbox` rows → GHL.
- Inbound webhook: `POST /webhooks/ghl?tenantId=&secret=` (per-tenant secret, verified; acks 200 immediately, processes async).
- ⚠️ **GHL marketplace app is in DRAFT.** You need to **finish the OAuth setup** (client id/secret, redirect/callback = `${PUBLIC_API_URL}` GHL callback) and get the app to a usable state so tenants can connect **and so the GHL-side workflows can be built**. The workflows, custom fields and tags to create are fully specified in **`docs/GHL_AUTOMATION_OWNER_HANDOFF.md`** (5 workflows: pre-clean reminder, job-complete, debt chase, payment-failed chase, lead handoff; 5 custom fields; 4 tags). Those are GHL-config tasks that depend on the app OAuth being live.
- **GoCardless / Stripe:**
- GoCardless: mandate + payment (`src/integrations/gocardless/`), webhook `/webhooks/gocardless` , connected via a stored access token (validated against the GC API).
- Stripe: **Connect OAuth already implemented** (`/settings/stripe/authorize` → `/callback`), webhook `/webhooks/stripe` , checkout links. Works once a Stripe **platform** account + `STRIPE_CLIENT_ID` / `STRIPE_SECRET_KEY` exist.
- **Zapier → RoundFlow cutover** for Mark's live GoCardless charges: see GHL handoff §6 (disable Zapier at the exact moment RoundFlow goes live to avoid double/missed charges).
- **Webhooks:** `/webhooks/stripe` , `/webhooks/gocardless` , `/webhooks/ghl` — all mounted with `express.raw` before JSON parsing (for signature/secret verification).
- **Storage:** Supabase Storage bucket `visit-photos` — visit before/after photos, technician avatars, chat attachments. Signed upload URLs via `src/lib/storage.ts` .
- **Push:** Expo (`src/mobile/services/push.ts`), token-gated; `createNotification` is the producer hook (see §7 — no producers wired yet).
- **PDF:** Gotenberg (`src/integrations/pdf/render.ts`) renders invoice HTML → PDF. See §5 for the prod caveat.
- **Frontend contract:** admin web + technician app. The exact endpoints the frontends need (and what's shipped vs. what the frontend must still wire) are in `docs/Handoffs/FRONTEND_GAPS_RESPONSE_2026-10-13.md` and `MOBILE_GAPS_RESPONSE_2026-10-13.md` . ⚠️ **The newest routes (messaging, invoice PDF, reports export, preferences, all mobile-gap routes) are NOT in `src/swagger.ts` — add OpenAPI entries when you get a chance (Schemathesis contract tests don't cover them yet).**

---

5. Environments & deployment

Local setup (do this during handover — don't just read it)

1. `git clone` the repo, `npm ci` .
2. `cp .env.example .env` and fill from a **dev Supabase project** (DATABASE_URL, DIRECT_URL, SUPABASE_*). Generate an `ENCRYPTION_KEY` : `openssl rand -hex 32` .

3. `npx prisma generate` **and** `npx prisma generate --schema prisma/tenant/schema.prisma`.
4. *(optional, for invoice PDFs)* run Gotenberg locally on a non-3000 port: `docker run -d --name gotenberg -p 3001:3000 gotenberg/gotenberg:8` and set `GOTENBERG_URL=http://localhost:3001`.
5. `npm run dev` (tsx watch). `npm test` to run the suite (208 tests). `npm run build` to produce `dist/`.

CI/CD

GitHub Actions, one workflow per env (`.github/workflows/deploy-{dev,qa,prod}.yml`), triggered by **push to the matching branch** (`dev` / `qa` / `prod`). Each: test → `prisma migrate deploy` (public) → build + push image to ECR → deploy. A `concurrency` group serializes deploys per env (added after a clobber incident — see §6).

DEV — live ✅

- EC2 box `51.20.108.133`. Container `roundflow-dev` on `:3000` behind nginx → `api-dev.roundflow.ai`. A **Gotenberg** sidecar runs on a shared `roundflow` docker network; the app gets `GOTENBERG_URL=http://gotenberg:3000` (both baked into `deploy-dev.yml`). Env in `/home/ec2-user/.env.dev`.
- Tenant migrations run at container startup (~60–100s cold start while they apply; nginx may 502 during that window — normal).

QA — exists but NOT IN USE ⚠️

- `deploy-qa.yml` is wired (push to `qa` → same EC2 box, container `roundflow-qa` on `:3001`, env `/home/ec2-user/.env.qa`, Gotenberg sidecar shared). **But there is no QA database, so QA is dormant.**
- **To activate QA:** 1. **Create a new Supabase project — on Mark Coates' email** (this is the QA system-of-record + auth). 2. Put its `DATABASE_URL`, `DIRECT_URL`, `SUPABASE_URL`, `SUPABASE_ISSUER`, `SUPABASE_SERVICE_ROLE_KEY` (and the rest of the env) into `/home/ec2-user/.env.qa` on the EC2 box, and set the QA `DATABASE_URL` / `DIRECT_URL` in the GitHub `environment: qa` secrets (for the migrate step). 3. Point a QA CORS origin in `FRONTEND_URL` and a QA API host as needed. 4. Push to the `qa` branch. Until then, treat QA as not real.

PROD — NOT SET UP ⚠️ ⚠️

- `deploy-prod.yml` deploys via **ECS** (not `docker run` like dev/qa): it renders a task definition and updates an ECS service, using GitHub secrets `ECS_TASK_DEFINITION`, `ECS_SERVICE`, `ECS_CLUSTER`.
- **Nothing is deployed to prod yet.** The entire Oct-2026 build (customer messaging, invoice PDF, reports export, per-user preferences, all the mobile-app gap fixes) is **dev-only**.
- Prod requires: (1) a prod **Supabase project** + full env, (2) an **ECS cluster + service + task definition**, (3) the prod GitHub secrets, (4) DNS/SSL for `api.roundflow.ai`.
- ⚠️ **What we added that will NOT work on prod as-is — Gotenberg (the invoice-PDF renderer):** on dev/qa we run Gotenberg as a sidecar via `docker run` on a shared docker network. **Prod is ECS, so that wiring does not apply.** On prod, Gotenberg must be added as a **sidecar container in the ECS task definition**, with the app container's `GOTENBERG_URL=http://localhost:3000` (ECS task containers share `localhost`). **Until that's done, `GET /invoices/:id/pdf` and `GET /invoices/preview/pdf` return `501` on prod.** (If anyone refers to "Heisenberg" — they mean **Gotenberg**.)

Jobs / cron (in-process, `node-cron`, `src/index.ts`)

- **Nightly visit generation** — `0 1 * * *` → `runVisitGeneration()` (creates visits from service plans for 4/6/8/12-week frequencies, honoring payment holds).

- **Outbox worker** — every 30s → `runOutboxWorker()` (drains `IntegrationOutbox` → GHL).
- **Startup** — `migrateAllTenantSchemas()` (applies pending tenant migrations to every schema).

---

6. Monitoring & operations

- **Logs:** `docker logs roundflow-dev` on the EC2 box (console logging). **No Sentry/structured logging yet** — worth adding.
- **Known incidents runbook:**
- **Deploy "succeeds" but old code runs (full-disk stale-image).** The 8 GB EC2 disk fills from accumulated images; `docker pull` then fails and — because the deploy script has **no `set -e`** — `docker run` silently falls back to the **stale local image**. Symptom: green deploy, but new routes/migrations missing. **Fix already added:** `docker image prune -f` before the pull. If it recurs: SSH in, `df -h /`, `docker image prune -f` (or `-a`). Verify a deploy landed by checking `docker exec roundflow-dev ls dist/prisma-tenant/migrations` and the `[migration] … applied` log lines — **not** by hitting `/mobile/*` (that returns 401 for any path because `requireAuth` runs before routing).
- **Deploy race.** Two near-simultaneous pushes could clobber `dev-latest`. Fixed with a `concurrency:` group in the workflows.
- **Cold-start 502 (~60–100s).** Per-tenant startup migrations delay `listen`; nginx returns 502 briefly after each deploy. Normal.
- **Third-party limits:** GHL API rate limits (handled with token refresh + retries), Expo push, Supabase plan quotas.

---

7. Current work status

- **Branches / PRs:** `dev` is the integration branch and is current. Oct-2026 feature branches were merged into `dev` and can be deleted. No critical open PRs.
- **Recently shipped (DEV only):**
- *Frontend gaps:* `POST /messages/send` + `/messages/bulk` (GHL, send-now), `GET /invoices/:id/pdf` + `/invoices/preview/pdf` (Gotenberg), `GET /reports/export` (CSV), per-user preferences (`GET/PATCH /auth/me/preferences`), CORS multi-origin fix, Gotenberg deploy wiring.
- *Mobile gaps:* access-issue enum + policy fixes, partial-price keep/restore, `canStart` / `blockReason`, `GET/PATCH /mobile/me` (+avatar/area), `GET /mobile/company`, `POST /mobile/support/reports`, revenue breakdowns, chat photo attachments, notifications + device registration + Expo push.
- **Known bugs / tech debt (honest list):**
- **Fixed this cycle:** technician resolution (`supabaseUserId` vs `Technician.profileId` = `Profile.id`); deploy stale-image.
- **Invoice "Send" is stubbed** — `POST /invoices/:id/send` flips status to `SENT` but **does not actually email/deliver** anything yet (`ponytail:` comment in `invoice.service.ts`).
- **Notification producers not wired** — `createNotification` exists; nothing calls it. Deciding *what events* notify (round assignment, payment, schedule) is a product call.
- **No OpenAPI/Swagger** for the newest routes.
- **Gotenberg not on prod** (§5), **QA not set up** (§5), **prod not set up** (§5).
- **Stale docs:** `ARCHITECTURE.md` overview says Railway; `PROGRESS.md` stale pre-2026-10-13; `RAILWAY_PUBLIC_DOMAIN` leftover.

- **Pending decisions / blockers:**
- **GHL marketplace app OAuth** must be finished (then the GHL workflows can be built — GHL handoff).
- Payment connect is blocked on real **Stripe/GoCardless accounts** (code is ready; Stripe OAuth exists).
- **Customer spreadsheet import** blocked on **Mark's Jobber template**.
- Mobile product decisions: access-issue auto-close, whether to expose mobile signup (recommend: no — invite-only), remove subscription/add-account screens, AI assistant.
- **Deadlines:** technician app target **15 Oct** — most mobile endpoints are done on dev; **critical gap: prod isn't set up**, so plan the prod bring-up now. Customer portal **mid-to-late Jan**.
- **Hardcoded/TODO:** search the codebase for `ponytail:` comments (deliberate shortcuts with their upgrade path) and `TODO`. Deploy workflows hardcode the ECR registry + EC2 flow.

---

8. Testing

- **Suite:** `npm test` (`vitest run`). 208 tests — service logic with mocked Prisma, plus some route tests (`supertest`) that **`vi.mock` the auth middleware**. CI runs tests **without app env**, so a test must not import a module that reads env at load (e.g. `requireAuth` needs `SUPABASE_ISSUER`) unless it mocks it — keep pure logic in `lib/` and test that (see `src/lib/preferences.ts`).
- **Coverage gaps:** no integration tests against a live DB; no OpenAPI contract coverage for new routes.
- **Test credentials (get from the team):** GHL sandbox, GoCardless sandbox, Stripe test mode, a Supabase dev login. A dev tenant exists on the dev Supabase project.
- **Manual QA:** ask Hamza for the manual verification flows he runs.

First-week checklist for the new dev

- [ ] Get all access in §1 (repo, AWS IAM, EC2 key, Supabase dev, GHL, GoCardless/Stripe, Google Cloud Console, password-manager secrets).
- [ ] Run the project locally (§5) and `npm test` green.
- [ ] Trace one request end-to-end: JWT → `requireTenantAccess` → a `/mobile` or `/settings` route → service → tenant schema.
- [ ] Read `ARCHITECTURE.md` , the GHL handoff, and the two gap-response docs.
- [ ] **Finish the GHL marketplace app OAuth** → unblock the GHL workflows.
- [ ] **Stand up QA** (Supabase project on Mark's email + `.env.qa`) and **plan the prod bring-up** (Supabase + ECS + **Gotenberg sidecar**).
- [ ] Confirm Supabase backups + do one test restore.